

LearnPulse AI

AI-Powered Learning Management System with Contextual AI Tutor

WEEK 4 IMPLEMENTATION REPORT

Base Domain Entities, Student & Teacher Profile Services, SecurityContext Ownership Enforcement, Academic Hierarchy (Subjects & Chapters), Global Exception Handling, & RBAC Content Protection

Project Name:	AI-Powered Learning Management System with Contextual AI Tutor
Document Title:	Week 4 Implementation Report (Domain Entities & User Core)
Official Phase:	WEEK 4 IMPLEMENTATION (Git Branch: week-4-implementation)
Technology Stack:	Spring Boot 3.2.5, Java 21, Spring Security 6, PostgreSQL 16, pgvector 0.8.2
Document Version:	1.0.0-FINAL
Report Date:	August 19, 2026
Verification Status:	COMPLETED & VERIFIED (BUILD SUCCESS — 100% Tests Pass)
Prepared For:	Project Mentor & Evaluation Committee

- **Project Title:** AI-Powered Learning Management System with Contextual AI Tutor
- **Document Title:** Week 4 Implementation Report
- **Phase:** WEEK 4 IMPLEMENTATION — Base Domain Entities & User Management Core
- **Focus:** Student & Teacher Profile Management, Academic Hierarchy (Subjects & Chapters), Global Exception Handling, & RBAC Content Protection
- **Technology Stack:** Spring Boot 3.2.5 + Java 21 + JWT 0.12.5 + Spring Security 6 + PostgreSQL 16 + pgvector 0.8.2 + Spring Data JPA
- **Version:** 1.0.0-FINAL
- **Date:** August 19, 2026
- **Status:** Completed, Audited, Built (BUILD SUCCESS), & Verified (100% Tests Pass — 22/22)
- **Prepared for:** Project Mentor & Evaluation Committee

1. WEEK 4 OVERVIEW

During **Week 4**, the engineering team implemented the **Base Domain Entities and User Management Core** on top of the established Week 2 Spring Boot infrastructure and Week 3 Spring Security/JWT foundation.

Week 4 introduces:

1. **User Profile Management Services & APIs:** Secure retrieval and transactional update of student (/api/student/profile) and teacher (/api/teacher/profile) profiles.
2. **SecurityContext Ownership Enforcement:** Derives user identity strictly from authenticated security principals, preventing horizontal privilege escalation or unauthorized profile modifications.
3. **Academic Hierarchy Domain Models:** Implemented JPA entities and repositories for Subject and Chapter establishing a 1-to-Many parent-child relationship.
4. **Subject & Chapter RESTful CRUD APIs:** Full lifecycle endpoints for subjects (/api/subjects) and chapters (/api/subjects/{subjectId}/chapters).
5. **RBAC Content Management Protection:** Restricted content modification (POST, PUT, DELETE) to ADMIN and TEACHER roles, blocking STUDENT users.
6. **Centralized Exception Infrastructure:** Integrated ResourceNotFoundException, constraint violation handling (HTTP 409 Conflict), and Bean Validation formatting into the standard ApiResponse wrapper.

2. WEEK 4 OBJECTIVES

The table below maps the mentor's explicit Week 4 specifications against the actual technical implementation and verification evidence:

Requirement	Implementation Status	Evidence / Verification
1. Student Profile Service & DTO	Completed	StudentService and StudentProfileDTO implemented with transaction boundaries.
2. Teacher Profile Service & DTO	Completed	TeacherService and TeacherProfileDTO implemented with transaction boundaries.
3. Student Profile APIs	Completed	GET & PUT /api/student/profile implemented in StudentProfileController.
4. Teacher Profile APIs	Completed	GET & PUT /api/teacher/profile implemented in TeacherProfileController.
5. Profile Ownership Protection	Completed	Identity derived strictly from @AuthenticationPrincipal User currentUser.

6. Subject JPA Entity & Repository	Completed	Subject.java entity and SubjectRepository created with unique constraints.
7. Chapter JPA Entity & Repository	Completed	Chapter.java entity and ChapterRepository created with subject foreign key.
8. Subject-Chapter 1-to-Many Relationship	Completed	Configured @OneToMany and @ManyToOne with CascadeType.ALL and orphanRemoval.
9. Subject CRUD REST APIs	Completed	Created SubjectController (POST, GET, PUT, DELETE /api/subjects).
10. Chapter CRUD REST APIs	Completed	Created ChapterController (POST, GET, PUT, DELETE under /api/chapters).
11. RBAC Content Protection	Completed	Content modifications restricted to ADMIN and TEACHER (@PreAuthorize).
12. Global Exception Handling	Completed	ResourceNotFoundException and constraint handling added to GlobalExceptionHandler.
13. OpenAPI / Swagger Documentation	Completed	Updated OpenAPI @Tag and @Operation annotations across all new controllers.
14. End-to-End Automated Testing	Completed	Executed 22/22 automated tests (mvn clean test **BUILD SUCCESS**).

3. STUDENT PROFILE MODULE OVERVIEW

The Student Profile module manages student-specific profile metadata (enrollmentNumber, department, firstName, lastName).

```

[ Client ] ---> GET /api/student/profile (Bearer Token)
      |
      v
[ StudentProfileController ] ---> @AuthenticationPrincipal User currentUser
      |
      v
[ StudentService.getStudentProfile() ] ---> UserProfileRepository.findById()
      |
      v
[ StudentProfileDTO ] <--- Map Entity to DTO <--- [ UserProfile Entity ]

```

4. TEACHER PROFILE MODULE OVERVIEW

The Teacher Profile module manages teacher-specific profile metadata (department, firstName, lastName).

```

[ Client ] ---> PUT /api/teacher/profile (Bearer Token + DTO Body)
      |
      v
[ TeacherProfileController ] ---> @AuthenticationPrincipal User currentUser
      |
      v
[ TeacherService.updateTeacherProfile() ] ---> Enforce Security Ownership
      |
      v
[ UserProfileRepository.save() ] ---> Return Updated TeacherProfileDTO

```

5. PROFILE DTO DESIGN

Profile information is communicated using dedicated DTOs to avoid exposing internal database entities:

StudentProfileDTO:

```
public class StudentProfileDTO {
    private UUID id;
    private UUID userId;
    private String email;
    @NotBlank private String firstName;
    @NotBlank private String lastName;
    private String enrollmentNumber;
    private String department;
}
```

TeacherProfileDTO:

```
public class TeacherProfileDTO {
    private UUID id;
    private UUID userId;
    private String email;
    @NotBlank private String firstName;
    @NotBlank private String lastName;
    private String department;
}
```

6. PROFILE SERVICE DESIGN

StudentService and TeacherService encapsulate transactional business logic:

- `@Transactional(readOnly = true)` for retrieval methods (`getStudentProfile`, `getTeacherProfile`).
- `@Transactional` for update methods (`updateStudentProfile`, `updateTeacherProfile`).
- Throws `ResourceNotFoundException` if a profile is missing.

7. DTO <-> ENTITY MAPPING

Mapping is handled cleanly in the service layer:

- **Entity to DTO:** Extracts entity attributes and maps to builder DTOs.
- **DTO to Entity:** Updates entity fields directly in transactional boundary without replacing foreign key relations.

8. PROFILE OWNERSHIP ENFORCEMENT

To prevent Student A from accessing or modifying Student B's profile:

1. Controllers inject `@AuthenticationPrincipal User currentUser`.
2. Services query `UserProfileRepository.findById(currentUser.getId())`.
3. Client-supplied IDs in request parameters or body are strictly ignored for current-user profile endpoints.

9. STUDENT PROFILE API DOCUMENTATION

- ****GET /api/student/profile**:**
- **Authorization:** `ROLE_STUDENT`

- **Response:** 200 OK $\rightarrow$ ApiResponse<StudentProfileDTO>
- ****PUT /api/student/profile**:**
- **Authorization:** ROLE_STUDENT
- **Request Body:** StudentProfileDTO
- **Response:** 200 OK $\rightarrow$ ApiResponse<StudentProfileDTO>

10. TEACHER PROFILE API DOCUMENTATION

- ****GET /api/teacher/profile**:**
- **Authorization:** ROLE_TEACHER
- **Response:** 200 OK $\rightarrow$ ApiResponse<TeacherProfileDTO>
- ****PUT /api/teacher/profile**:**
- **Authorization:** ROLE_TEACHER
- **Request Body:** TeacherProfileDTO
- **Response:** 200 OK $\rightarrow$ ApiResponse<TeacherProfileDTO>

11. SUBJECT ENTITY DESIGN

The Subject entity models academic subjects:

```
@Entity
@Table(name = "subjects")
public class Subject {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false, unique = true)
    private String name;

    @Column(nullable = false, unique = true)
    private String code;

    @Column(length = 1000)
    private String description;

    @OneToMany(mappedBy = "subject", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Chapter> chapters = new ArrayList<>();
}
```

12. CHAPTER ENTITY DESIGN

The Chapter entity models chapters under a parent subject:

```

@Entity
@Table(name = "chapters", uniqueConstraints = {
    @UniqueConstraint(name = "uk_subject_chapter_number", columnNames = {"subject_id", "chapter_number"})
})
public class Chapter {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false)
    private String title;

    @Column(name = "chapter_number", nullable = false)
    private Integer chapterNumber;

    @Column(length = 1000)
    private String description;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "subject_id", nullable = false)
    private Subject subject;
}

```

13. SUBJECT-CHAPTER RELATIONSHIP

- **Cardinality:** 1 Subject to Many Chapters.
- **Cascade Behavior:** `CascadeType.ALL` with `orphanRemoval = true`. Deleting a Subject automatically deletes its associated Chapter entities.
- **Navigation:** Bidirectional JPA mapping.

14. DATABASE CONSTRAINTS

1. **Subject Code Uniqueness:** `UK_subject_code` (e.g. CS101).
2. **Subject Name Uniqueness:** `UK_subject_name`.
3. **Chapter Number Uniqueness per Subject:** `uk_subject_chapter_number` composite constraint on (`subject_id`, `chapter_number`).
4. **Foreign Key Integrity:** `FK_chapter_subject` ensures no orphaned chapters exist.

15. SUBJECT CRUD API DOCUMENTATION

- `POST /api/subjects` $\rightarrow$ Create Subject (ADMIN, TEACHER) $\rightarrow$ 201 Created
- `GET /api/subjects` $\rightarrow$ List All Subjects (Authenticated) $\rightarrow$ 200 OK
- `GET /api/subjects/{id}` $\rightarrow$ Get Subject Details (Authenticated) $\rightarrow$ 200 OK
- `PUT /api/subjects/{id}` $\rightarrow$ Update Subject (ADMIN, TEACHER) $\rightarrow$ 200 OK
- `DELETE /api/subjects/{id}` $\rightarrow$ Delete Subject (ADMIN, TEACHER) $\rightarrow$ 200 OK

16. CHAPTER CRUD API DOCUMENTATION

- `POST /api/subjects/{subjectId}/chapters` $\rightarrow$ Create Chapter (ADMIN, TEACHER) $\rightarrow$ 201 Created
- `GET /api/subjects/{subjectId}/chapters` $\rightarrow$ List Chapters for Subject (Authenticated) $\rightarrow$ 200 OK
- `GET /api/chapters/{id}` $\rightarrow$ Get Chapter Details (Authenticated) $\rightarrow$ 200 OK
- `PUT /api/chapters/{id}` $\rightarrow$ Update Chapter (ADMIN, TEACHER) $\rightarrow$ 200 OK

- DELETE /api/chapters/{id} → Delete Chapter (ADMIN, TEACHER) → 200 OK

17. RBAC CONFIGURATION

Content management write operations (POST, PUT, DELETE) are guarded with `@PreAuthorize("hasAnyRole('ADMIN', 'TEACHER'))"`. STUDENT users attempting content modifications receive HTTP 403 Forbidden.

18. GLOBAL EXCEPTION HANDLING DESIGN

GlobalExceptionHandler centrally formats exceptions into ApiResponse:

- ResourceNotFoundException → HTTP 404 NOT_FOUND
- ApiException → Dynamic HTTP status
- AccessDeniedException → HTTP 403 FORBIDDEN
- DataIntegrityViolationException → HTTP 409 CONFLICT
- MethodArgumentNotValidException → HTTP 400 BAD_REQUEST

19. VALIDATION STRATEGY

Bean Validation annotations (`@NotBlank`, `@Size`, `@Min`, `@NotNull`) validate incoming DTOs before controller execution. Validation errors return HTTP 400 Bad Request with structured error lists.

20. SWAGGER/OPENAPI DOCUMENTATION

OpenAPI 3.0 annotations (`@Tag`, `@Operation`) document all Week 4 endpoints. Swagger UI is accessible at `/swagger-ui.html`.

21. TESTING STRATEGY

Automated test suites cover:

1. Student & Teacher profile retrieval, updates, and ownership isolation (`StudentTeacherProfileIntegrationTest`).
2. Subject & Chapter CRUD lifecycles, relationship integrity, and duplicate code rejection (`AcademicHierarchyIntegrationTest`).
3. Security boundary checks (STUDENT role forbidden from content modification).
4. Regression verification of Week 2 status API and Week 3 auth tests.

22. API TESTING RESULTS

Executing Maven test suite:

```
.tools/apache-maven-3.9.6/bin/mvn clean test
```

Execution Summary:

```
[INFO] Running com.learnpulse.backend.LearningAssistantApplicationTests (1 test)
[INFO] Running com.learnpulse.backend.security.JwtProviderTest (3 tests)
[INFO] Running com.learnpulse.backend.security.SecurityRbacIntegrationTest (7 tests)
[INFO] Running com.learnpulse.backend.academic.AcademicHierarchyIntegrationTest (5 tests)
[INFO] Running com.learnpulse.backend.controller.BaseStatusControllerTest (1 test)
[INFO] Running com.learnpulse.backend.profile.StudentTeacherProfileIntegrationTest (5 tests)
[INFO]
[INFO] Results:
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

23. DATABASE CONSTRAINT VERIFICATION

- 1. **Unique Code Check:** Verified CS101 duplicate creation throws HTTP 409 Conflict.
- 2. **Unique Chapter Number Check:** Verified duplicate chapter numbers under the same subject throw HTTP 409 Conflict.
- 3. **Foreign Key Deletion:** Verified deleting a Subject cleanly cascades deletion to all child Chapter records.

24. FILES CREATED AND MODIFIED

File Path	Action	Technical Purpose
StudentProfileDTO.java	Created	DTO for student profile API requests and responses
TeacherProfileDTO.java	Created	DTO for teacher profile API requests and responses
StudentService.java	Created	Service executing student profile retrieval & updates
TeacherService.java	Created	Service executing teacher profile retrieval & updates
StudentProfileController.java	Created	REST controller for /api/student/profile
TeacherProfileController.java	Created	REST controller for /api/teacher/profile
ResourceNotFoundException.java	Created	Runtime exception mapping to HTTP 404 NOT_FOUND
Subject.java	Created	JPA Entity for academic subjects
Chapter.java	Created	JPA Entity for academic chapters
SubjectRepository.java	Created	Spring Data JPA repository for Subject
ChapterRepository.java	Created	Spring Data JPA repository for Chapter
SubjectDTO.java	Created	DTO for subject responses
CreateSubjectRequest.java	Created	Validation DTO for subject creation and updates
ChapterDTO.java	Created	DTO for chapter responses
CreateChapterRequest.java	Created	Validation DTO for chapter creation and updates
SubjectService.java	Created	Service executing subject CRUD operations & constraint validation
ChapterService.java	Created	Service executing chapter CRUD operations & relationship checks
SubjectController.java	Created	REST controller for /api/subjects
ChapterController.java	Created	REST controller for /api/chapters
GlobalExceptionHandler.java	Modified	Updated with ResourceNotFoundException and DataIntegrityViolationException handlers
StudentTeacherProfileIntegrationTest.java	Created	Integration tests for profile APIs and ownership protection
AcademicHierarchyIntegrationTest.java	Created	Integration tests for Subject/Chapter CRUD & RBAC protection
WEEK_4_IMPLEMENTATION_REPORT.md	Created	Official Week 4 Implementation Report Markdown source
WEEK_4_IMPLEMENTATION_REPORT.pdf	Created	Official Week 4 Implementation Report PDF document

25. REQUIREMENT-BY-REQUIREMENT CHECKLIST

Mentor Requirement	Target Day	Status	Empirical Evidence
Implement StudentService & TeacherService	Day 19	Completed	StudentService.java and TeacherService.java created and tested.
Implement StudentProfileDTO & TeacherProfileDTO	Day 19	Completed	DTO classes created with Bean Validation annotations.
Implement StudentProfileController (GET/PUT /api/student/profile)	Day 20	Completed	RestController created and verified with MockMvc.
Implement TeacherProfileController (GET/PUT /api/teacher/profile)	Day 20	Completed	RestController created and verified with MockMvc.
Enforce SecurityContext profile ownership	Day 20	Completed	Identity derived strictly from @AuthenticationPrincipal User currentUser.
Implement Subject and Chapter JPA entities	Day 21	Completed	Entities created in com.learnpulse.backend.entity.
Configure Subject-Chapter 1-to-Many relationship	Day 21	Completed	@OneToMany and @ManyToOne configured with cascade delete.
Implement SubjectRepository & ChapterRepository	Day 21	Completed	Spring Data JPA interfaces created and tested.
Implement Subject CRUD APIs (/api/subjects)	Day 22	Completed	SubjectController.java created with full lifecycle operations.
Implement Chapter CRUD APIs (/api/chapters)	Day 22	Completed	ChapterController.java created with full lifecycle operations.
Restrict content management to ADMIN and TEACHER roles	Day 22	Completed	@PreAuthorize("hasAnyRole('ADMIN', 'TEACHER')") applied and verified.
Extend GlobalExceptionHandler with 404 & 409 handlers	Day 23	Completed	ResourceNotFoundException and constraint handling added.
Perform comprehensive automated testing	Day 24	Completed	Executed 22/22 tests (mvn clean test **BUILD SUCCESS**).

26. ITEMS INTENTIONALLY NOT IMPLEMENTED

In strict compliance with the mentor's Week 4 specification, the following business domain modules were **intentionally NOT implemented**:

- **Course & Content Management**: No courses, course enrollments, lecture notes, or PDF document uploads.
- **Document Processing Engine**: No text chunking, PDF page extraction, or Apache Tika/PDFBox parsing pipelines.
- **Assessment Engine**: No quizzes, questions, options, quiz attempts, or grading logic.
- **AI Subsystem & RAG Pipeline**: No passage embeddings, pgvector vector similarity searches, prompt assembly, or LLM streaming APIs.
- **User Dashboards**: No student, teacher, or admin UI dashboards or business analytics.

27. SCOPE / DEVIATION DISCLOSURE

- **No Unrequested Additions:** No unrequested technologies, frameworks, or business modules were added.
- **No Architecture Redesign:** All work extends the established Week 2 backend infrastructure and Week 3 security framework.

NOTE:

No Material Deviations: No material deviations from the mentor's Week 4 specification were made.

28. KNOWN LIMITATIONS

1. **In-Memory Subject Deletion Cascade:** Subject deletion cascades to child chapters at the database level. For large hierarchies, soft-deletion (deactivation) can be considered in future feature phases.
 2. **Profile Attachment Storage:** Profile data currently handles structured text fields (department, enrollmentNumber). Profile avatar image uploads belong to future file storage phases.
-

29. WEEK 4 COMPLETION SUMMARY

Week 4 has successfully established the core domain entities and user management capabilities for the AI-Powered Learning Management System. The implementation includes student and teacher profile services, SecurityContext ownership enforcement, Subject and Chapter JPA entities, Spring Data JPA repositories, RESTful CRUD controllers, RBAC content protection for ADMIN and TEACHER roles, and enhanced global exception handling.

All features have been verified via 22 automated unit and integration tests, achieving ****100% build success (BUILD SUCCESS)**** and zero test failures.

"Only the functionality specified in the Week 4 mentor specification was implemented. No future business or AI modules were implemented."

End of Official Week 4 Implementation Report.